Hub Rejection and Convergence in the lux Commit-on-Idle Discipline: a Z Specification

Formal model of the honour-forever divergence and its exclusion

July 2026

1 Introduction

The commit-on-idle reconciliation of `input_text_reconciliation.tex` governs a field whose value is Hub-authoritative and replicated to a Display. Its carrier `[VALUE]` stands for whatever a field holds — text, a float, an RGBA tuple — and its `HubEcho` operation installs the committed value at the Hub unconditionally. That unconditional install encodes an assumption the present document makes explicit: *every value a field commits is one the Hub will accept.* The carrier is, in effect, the set of Hub-accepted values.

The assumption is false for a numeric field. An `input_float` widget does not clamp, and the renderer's own projection into range, `elem.clamped`, leaves a value non-finite when the field is unbounded on the offending side: a typed `1e400` becomes `+inf` and passes through. The field commits that value; the Hub's `apply_patch` re-checks range *and* finiteness and raises `ValueError`; the firing host logs the error and does not re-push. The Hub value therefore never advances to the committed value.

The arbiter's optimistic echo is what turns a rejected commit into a permanent fault. On commit the arbiter records the committed value and the Hub value observed at commit time, and while the field is idle it returns the committed value until the Hub value moves *off* that commit-time value — the moment it treats as the echo landing. When the Hub rejects the commit, the Hub value never moves; the arbiter's forget branch never fires; and the Display honours the rejected value for the rest of the field's life while the Hub holds the old one. This is the *honour-forever divergence*. It surfaced twice on `input_number` — once for an out-of-range value, once for a non-finite one — and that recurrence is why the boundary is modelled here rather than patched a third time.

The correction is a single discipline: *a field commits only a value the Hub will accept.* The renderer's pre-commit projection is widened from a range clamp to a total sanitiser onto the accepted domain, so a non-finite or out-of-range entry is repaired before it is committed, and the value that travels to the Hub is always one `apply_patch` installs. The Hub then always echoes, the forget branch always eventually fires, and the display converges.

This document distinguishes the Hub-accepted subset from an arbitrary committed value, adds an explicit Hub-reject transition, and states the convergence property the defect violates. Over a small bounded carrier ProB enumerates every interleaving of edit, commit, echo, and reject, and either confirms convergence or produces the exact honour-forever trace.

2 Basic Types

$[VALUE]$

`VALUE` is the set of values the field may hold and the widget may commit. Crucially it is *not* restricted to Hub-accepted values: the widget can produce a value the Hub refuses, and the whole point of this model is that such a value can be committed. Two values suffice to exhibit the divergence — an accepted starting value and one value the Hub rejects; three widen the interleaving without adding a reachable fault. The carrier is bounded by ProB's `DEFAULT_SETSIZE`.

3 Free Types and Constants

$ZBOOL ::= ztrue \mid zfalse$

A two-valued flag written as a free type rather than the toolkit `bool`, so ProB animates it as a small enumerated set.

$$
\begin{array}{|l}
v0 : VALUE \\
accepted : \mathbb{P}\ VALUE \\
\hline
v0 \in accepted
\end{array}
$$

`accepted` is the Hub-accepted domain: the set of values `apply_patch` installs rather than rejecting. For a numeric field it is the finite values within `[min, max]`; the model leaves it an arbitrary subset so ProB checks every shape of boundary, including the one where only the start value is accepted. `v0` is that start value — both the initial display and the initial Hub value — and it must be accepted, since the Hub holds it. A value in `VALUE` but outside `accepted` is one the widget can commit and the Hub will reject; with `DEFAULT_SETSIZE 2` and `accepted` taken as {`v0`}, the other value is exactly such a one.

4 State

The state is flat: one schema, eight components. Its predicate carries only structural coherence — that a field can defer or be mid-edit only while focused. That holds in every design, correct or buggy, so it constrains rather than tests. The convergence property (Section 9) is deliberately *absent* here, so a divergent state stays reachable and ProB can find it.

Field

$$
\begin{array}{|l}
disp : VALUE \\
hub : VALUE \\
focused : ZBOOL \\
editing : ZBOOL \\
edited : ZBOOL \\
pending : ZBOOL \\
committed : VALUE \\
diverged : ZBOOL \\
\hline
editing = ztrue \Rightarrow focused = ztrue \\
edited = ztrue \Rightarrow focused = ztrue
\end{array}
$$

`disp` is the value the Display renders — the arbiter's `resolve` output. `hub` is the Hub-authoritative value. `focused`, `editing` (the defer flag) and `edited` (the real-edit witness) carry the honour/defer discipline unchanged from the parent model, kept so the convergence question is asked over a faithful edit cycle rather than an abstract one.

`pending` marks the echo-latency window: set from a commit, and cleared only when the Hub adopts the committed value. While it holds, `resolve` returns `committed`, so the display shows the committed value. `committed` is the value most recently committed — possibly one outside `accepted`. `diverged` is pure bookkeeping for the invariant: it latches `ztrue` the moment the Hub rejects an outstanding commit, the event after which the display honours a value the Hub will never hold.

5 Initialization

Init

$$
\begin{array}{|l}
Field' \\
\hline
disp' = v0 \\
hub' = v0 \\
focused' = zfalse \\
editing' = zfalse \\
edited' = zfalse \\
pending' = zfalse \\
committed' = v0 \\
diverged' = zfalse
\end{array}
$$

A single initial state: the field unfocused and idle, display and Hub both on the accepted value `v0`, no commit outstanding, nothing diverged.

6 The Edit Cycle

Focusing begins a fresh session and, following the corrected parent design, does *not* begin deferring: a focused-but-unedited field still honours the Hub.

Focus

$\Delta Field$

$focused = zfalse$
$focused' = ztrue$
$editing' = zfalse$
$edited' = zfalse$
$disp' = disp$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$diverged' = diverged$

A keystroke changes the text and begins deferring. The typed value `e?` ranges over the whole of `VALUE` — including values outside `accepted`: this is where the user types `1e400`, the value the Hub will later reject. Modelling `e?` as unrestricted is what lets the widget present the Hub with a value it refuses.

Keystroke

$\Delta Field$
$e? : VALUE$

$focused = ztrue$
$editing' = ztrue$
$edited' = ztrue$
$disp' = e?$
$focused' = focused$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$diverged' = diverged$

A commit is the deactivation of an edited field. It records the committed value and opens the echo-latency window; the Hub is not updated here. The guard $\texttt{disp} \in \texttt{accepted}$ is the *fix*: the field commits only a value the Hub will accept. It models the renderer's total pre-commit sanitiser — a non-finite or out-of-range entry is repaired into `accepted` before this point, so the committed value is always installable. Deleting the guard is the defect (Section 11): the widget then commits whatever it displays, rejected values included.

Commit

$\Delta Field$

$focused = ztrue$
$editing = ztrue$
$disp \in accepted$
$committed' = disp$
$pending' = ztrue$
$focused' = zfalse$
$editing' = zfalse$
$edited' = zfalse$
$disp' = disp$
$hub' = hub$
$diverged' = diverged$

A field may also lose focus without an edit. This commits nothing.

BlurNoEdit

$\Delta Field$

$focused = ztrue$
$editing = zfalse$
$focused' = zfalse$
$edited' = zfalse$
$editing' = zfalse$
$disp' = disp$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$diverged' = diverged$

7 The Hub Response

A committed value reaches the Hub, which either installs it or rejects it. The two outcomes are the two operations here, and they partition on whether the committed value is accepted.

`HubEchoAccept` is the delayed echo of an *accepted* commit: the Hub adopts the committed value and the echo-latency window closes. From here an honour frame renders `hub`, which now equals `committed`: the optimistic display and the real Hub value have converged.

HubEchoAccept

$\Delta Field$

$pending = ztrue$
$committed \in accepted$
$hub' = committed$
$pending' = zfalse$
$disp' = disp$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$committed' = committed$
$diverged' = diverged$

`HubReject` is the delayed *refusal* of a commit the Hub will not install: the committed value is outside `accepted`, so `apply_patch` raises and the firing host swallows the error. The Hub value does not move, and the echo-latency window does not close — `pending` stays set, so `resolve` keeps returning the rejected value. The operation latches `diverged`: from this state the display honours a value the Hub will never hold, and no operation can carry the Hub to it, because only `HubEchoAccept` advances `hub` and it is disabled while the committed value is unaccepted.

HubReject

$\Delta Field$

$pending = ztrue$
$committed \notin accepted$
$diverged' = ztrue$
$hub' = hub$
$pending' = pending$
$disp' = disp$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$committed' = committed$

8 The Render Frames

A frame in which the field is not deferring honours: it drives the display to the value the field is authoritative for — the committed value while a commit is in flight, the raw Hub value otherwise. This is the `resolve` rule: it is what makes the display show the committed value throughout the window, and hence what makes a rejected commit visible for as long as the window stays open.

IdleHonour

$\Delta Field$

$focused = zfalse$
$editing = zfalse$
$disp' = \textbf{if}\ pending = ztrue\ \textbf{then}\ committed\ \textbf{else}\ hub$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$diverged' = diverged$

FocusedHonour

$\Delta Field$

$focused = ztrue$
$editing = zfalse$
$disp' = \textbf{if}\ pending = ztrue\ \textbf{then}\ committed\ \textbf{else}\ hub$
$focused' = focused$
$editing' = editing$
$edited' = edited$
$hub' = hub$
$pending' = pending$
$committed' = committed$
$diverged' = diverged$

A deferring field — `editing` set — honours nothing: its display is the buffer, moved only by a further keystroke. That case is the absence of an honour frame while `editing` holds. It is what keeps a step always available: an unfocused field can `IdleHonour`; a focused non-deferring field can `FocusedHonour`; a deferring field is focused, so it can `Keystroke`. A step is therefore enabled in every reachable state, and the machine cannot deadlock (Section 9, invariant 3).

9 Invariants

Three properties must hold across all reachable states. None is placed in the `Field` schema: a predicate placed there would guard every successor, silently disabling any operation that would break it and masking the very defect the model exists to catch.

Invariant 1 — convergence (no honour-forever). Whenever a commit is outstanding, the outstanding value is one the Hub will accept. Then the echo can always land: `HubEchoAccept` is enabled, the window closes, and the display converges to the Hub. The negation — a commit outstanding whose value the Hub will refuse — is the honour-forever state, in which `resolve` returns the committed value on every frame while no operation can carry the Hub to it. The invariant is that this state is never reached:

$$pending = ztrue \Rightarrow committed \in accepted.$$

Invariant 2 — rejection never occurs. Equivalently, and as a named event: the Hub never rejects an outstanding commit, so the divergence latch never fires:

$$diverged = zfalse.$$

The two forms coincide. `HubReject` is enabled exactly when invariant 1's negation holds, so if invariant 1 holds on every reachable state then `HubReject` never fires and `diverged` stays clear; and if invariant 1's negation is reachable then `HubReject` fires from that state and latches `diverged`. Invariant 1 is the property; invariant 2 is the witness that makes the offending trace end on a single named transition.

Invariant 3 — deadlock freedom. No reachable state leaves the machine unable to progress. The honour frames cover the not-deferring case; a deferring field is focused and can `Keystroke`; so a step is always enabled.

10 Verification

Type-checking is a `fuzz` pass:

```
fuzz -t docs/reconciliation_hub_reject.tex
```

Invariants 1 and 2 are state properties, checked by asking ProB whether their negation is reachable. A goal not found means the invariant holds on every reachable state:

```
# Invariant 1 -- convergence (must report: goal NOT found)
probcli docs/reconciliation_hub_reject.tex -model_check -nodead \
  -goal "pending=ztrue & committed /: accepted" -p DEFAULT_SETSIZE 2

# Invariant 2 -- rejection never occurs (must report: goal NOT found)
probcli docs/reconciliation_hub_reject.tex -model_check -nodead \
  -goal "diverged=ztrue" -p DEFAULT_SETSIZE 2

# Inherited coherence -- defer only after a real edit (must report: NOT found)
probcli docs/reconciliation_hub_reject.tex -model_check -nodead \
  -goal "editing=ztrue & edited=zfalse" -p DEFAULT_SETSIZE 2
```

Invariant 3 is the deadlock check over the full reachable state space:

```
# Invariant 3 -- deadlock freedom (must report: no deadlock, no invariant viol.)
probcli docs/reconciliation_hub_reject.tex -model_check -p DEFAULT_SETSIZE 2
```

All return the same verdict at `DEFAULT_SETSIZE 3`: the extra value widens the interleaving of a second edit against an in-flight commit without adding a reachable violation. Because `accepted` is left underspecified, ProB discharges each goal over *every* valuation of the accepted domain that contains `v0` — so convergence is proved not for one boundary but for all.

11 Fidelity: reproducing the honour-forever divergence

A model that cannot reproduce the known defect proves nothing. The defect is the corrected specification with the commit guard removed: strike `disp` $\in$ `accepted` from `Commit`, so the field commits whatever it displays, rejected values included. Nothing else changes. Under `DEFAULT_SETSIZE 2`, take `accepted` = $\{$`v0`$\}$, and write $v_0 = v0$ for the accepted start value and v_r for the other, rejected, value.

With the guard gone, both convergence goals turn from *not found* to *found*. The shortest trace to `diverged = ztrue`:

1. `Init`: `disp = hub = committed` = v_0, unfocused, idle, `pending = zfalse`, `diverged = zfalse`.
2. `Focus`: `focused = ztrue`, still not editing.
3. `Keystroke(`v_r`)`: the user types the value the Hub will reject. `disp` = v_r, `editing = ztrue`.

4. `Commit` (unguarded): `committed` = v_r, `pending` = `ztrue`, `hub` = v_0, `focused` = `zfalse`, `editing` = `zfalse`. A commit is now outstanding for a value outside `accepted` — invariant 1's negation already holds: `pending` = `ztrue` and `committed` = $v_r \notin$ `accepted`.

5. `IdleHonour`: `pending` holds, so `disp'` = `committed` = v_r. The Display shows the rejected value.

6. `HubReject`: `committed` = $v_r \notin$ `accepted`, so the Hub refuses. `hub` stays v_0, `pending` stays `ztrue`, and `diverged` latches `ztrue`.

From the state after step 6 the divergence is permanent. `HubEchoAccept` is disabled (`committed` $\notin$ `accepted`); `HubReject` only re-latches; every `IdleHonour` rewrites `disp` = `committed` = v_r. The Display honours v_r forever, the Hub holds v_0 forever, and the arbiter's forget branch — which fires only when `hub` moves off the commit-time value — never fires because `hub` never moves. This is exactly the field-observed fault: a value typed past the Hub's boundary, committed, echoed nowhere, and displayed indefinitely.

Under the corrected design the guard `disp` $\in$ `accepted` makes step 4 impossible for v_r: `Commit` is disabled while the display holds a rejected value, so the user must repair the entry (a further `Keystroke` into `accepted`) or blur. `committed` $\in$ `accepted` therefore holds throughout every window; `HubReject` is dead; `HubEchoAccept` always eventually fires; and the reachability search confirms `pending = ztrue & committed /: accepted` and `diverged = ztrue` are both *not found*, for every valuation of `accepted`.

12 From the Model to the Code

The model's commit guard is one change to the numeric renderer. Today `InputNumberRenderer` projects a raw entry through `elem.clamped` before committing; `clamped` bounds the value into `[min, max]` but does nothing to a non-finite value on an unbounded side, so `+inf` survives to the commit and the Hub rejects it. The corrected projection is a *total sanitiser* onto the accepted domain: it maps every widget value — non-finite included — to a finite in-range value before the value is observed, committed, or fired. A non-finite entry collapses to a bound (or to the field's current value when the field is unbounded), an out-of-range entry clamps, and the value that reaches the Hub is one `apply_patch` installs without raising.

Concretely, the pre-commit projection must satisfy, for every widget value `r`, `sanitise(r)` $\in$ `accepted` — the model's guard discharged as a totality obligation on the sanitiser rather than as a runtime check on the commit. With that obligation met, `apply_patch`'s boundary re-check becomes a never-tripped invariant, the arbiter's forget branch always eventually fires, and no rejected value can be honoured.

The obligation is exactly the one the parent model discharged *structurally* for the colour carrier — a hex string cannot encode a non-finite value, so its parse is total onto `[0,1]` — and *by validation* for the slider, whose `validate` carries a `math.isfinite` guard because raw floats cross the wire. The numeric field is the slider's case: raw numbers cross the wire, so finiteness must be enforced, and the natural place is the same pre-commit projection that already enforces range.

13 Scope and Relation to the Parent Model

This specification isolates the *convergence* concern and takes the parent model's honour/defer/clobber concerns as given. It keeps the edit cycle (`Focus`, `Keystroke`, `Commit`, the two honour frames) so that the convergence question is asked over a faithful commit-echo loop, but it does not restate the durability (`lost`) or no-clobber (`clobbered`) invariants; those are proved in `input_text_reconciliation.tex` and are unaffected by the commit guard, which only narrows the set of committable values.

Two modelling choices bound the claim. First, `accepted` is static: the Hub's accepted domain does not change under the field's own operations, which matches a field whose `min`/`max`/finiteness rule is fixed for the life of the element. A field whose bounds an agent narrows *while a commit is in flight* could reject a previously-acceptable commit; that is a distinct interleaving, outside this model, and would be caught by the same invariant were `accepted` made mutable. Second, a single `committed` component admits only one outstanding commit, as in the parent model; the honour-forever fault needs only one, and modelling a queue would add interleaving the fault does not require. Both are consistent with the parent model's single-slot, value-equality reconciliation.